Linux Application Development - Day 3

File Systems and I/O Operations

Instructor: [Your Name]

Duration: 8 hours (including labs)

Welcome to Day 3!

Previous Days Recap

✅ **Day 1:** Build tools, libraries, Make, Git

✅ **Day 2:** Debugging (GDB, Valgrind), system calls, memory

Today's Focus

📁 **File systems** - VFS, inodes, ext4 architecture

📄 **File I/O** - open, read, write, seek APIs

🚀 **Advanced I/O** - mmap, scatter/gather, async I/O

🔒 **File locking** - Coordination between processes

Day 3 Agenda

Time	Topic
09:00-10:30	File Systems and VFS Architecture
10:30-10:45	Break
10:45-12:00	File I/O APIs and Best Practices
12:00-13:00	Lunch
13:00-14:30	Advanced File Operations
14:30-14:45	Break
14:45-16:00	Memory-Mapped I/O and File Locking
16:00-17:00	Hands-On Lab

Section 1: Linux File Systems

"Everything is a File"

Unix Philosophy

- **Regular files** - data storage
- **Directories** - file containers
- **Devices** - `/dev/sda`, `/dev/tty`
- **Pipes** - IPC mechanisms
- **Sockets** - network communication
- **Symlinks** - shortcuts

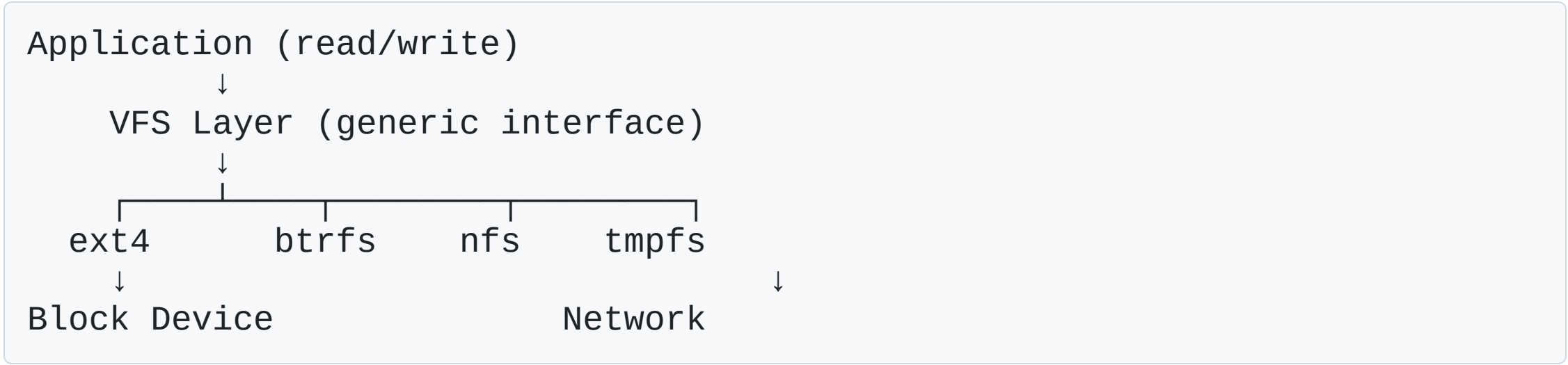
Uniform interface: open, read, write, close

File Types in Linux

```
# List files with types
ls -lF

- = regular file
d = directory
l = symbolic link
c = character device
b = block device
p = named pipe (FIFO)
s = socket

# File command
file /dev/sda    # block special
file /bin/bash   # ELF 64-bit executable
file document.pdf # PDF document
```

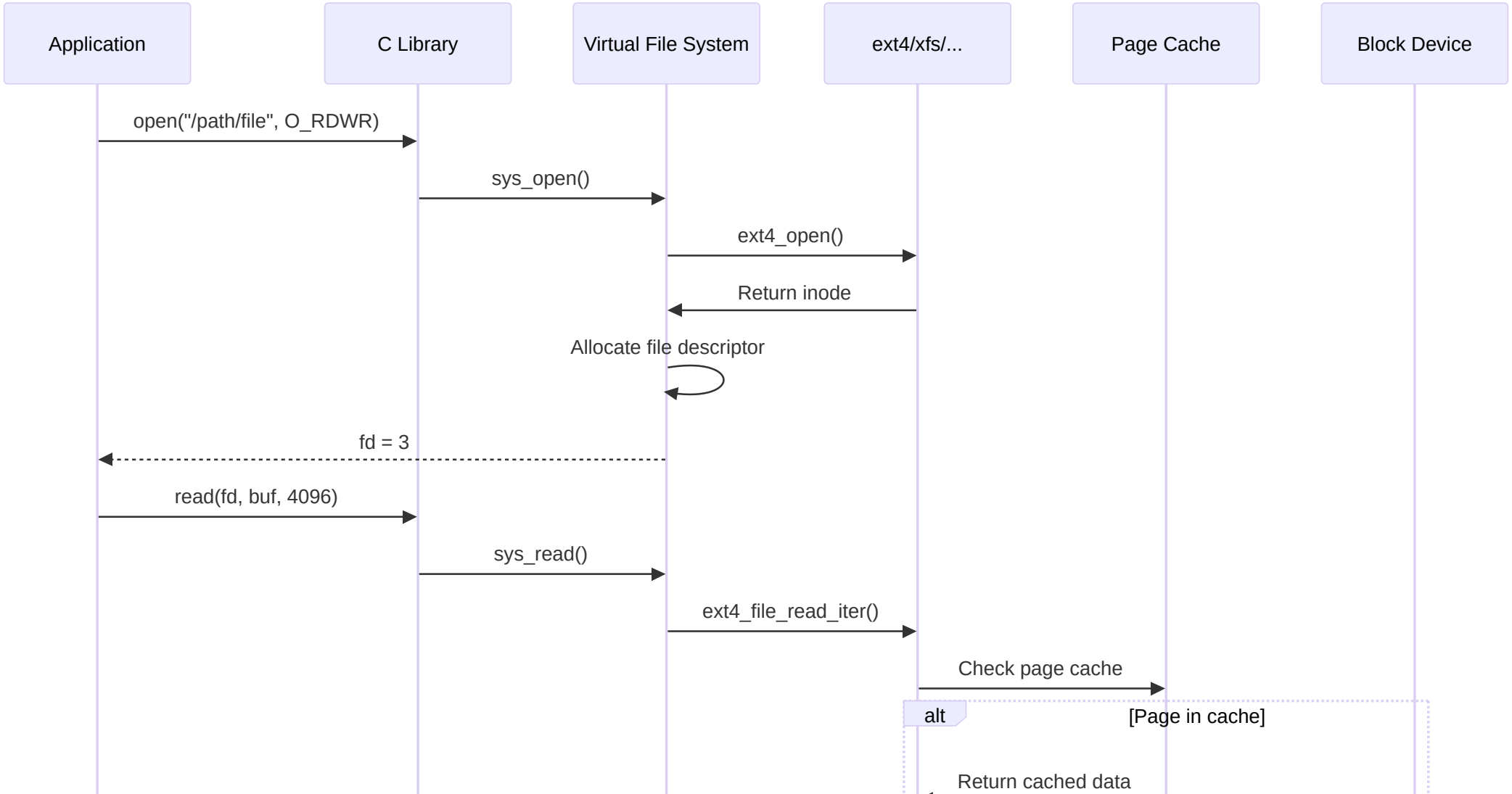
Virtual File System (VFS)

Abstraction Layer

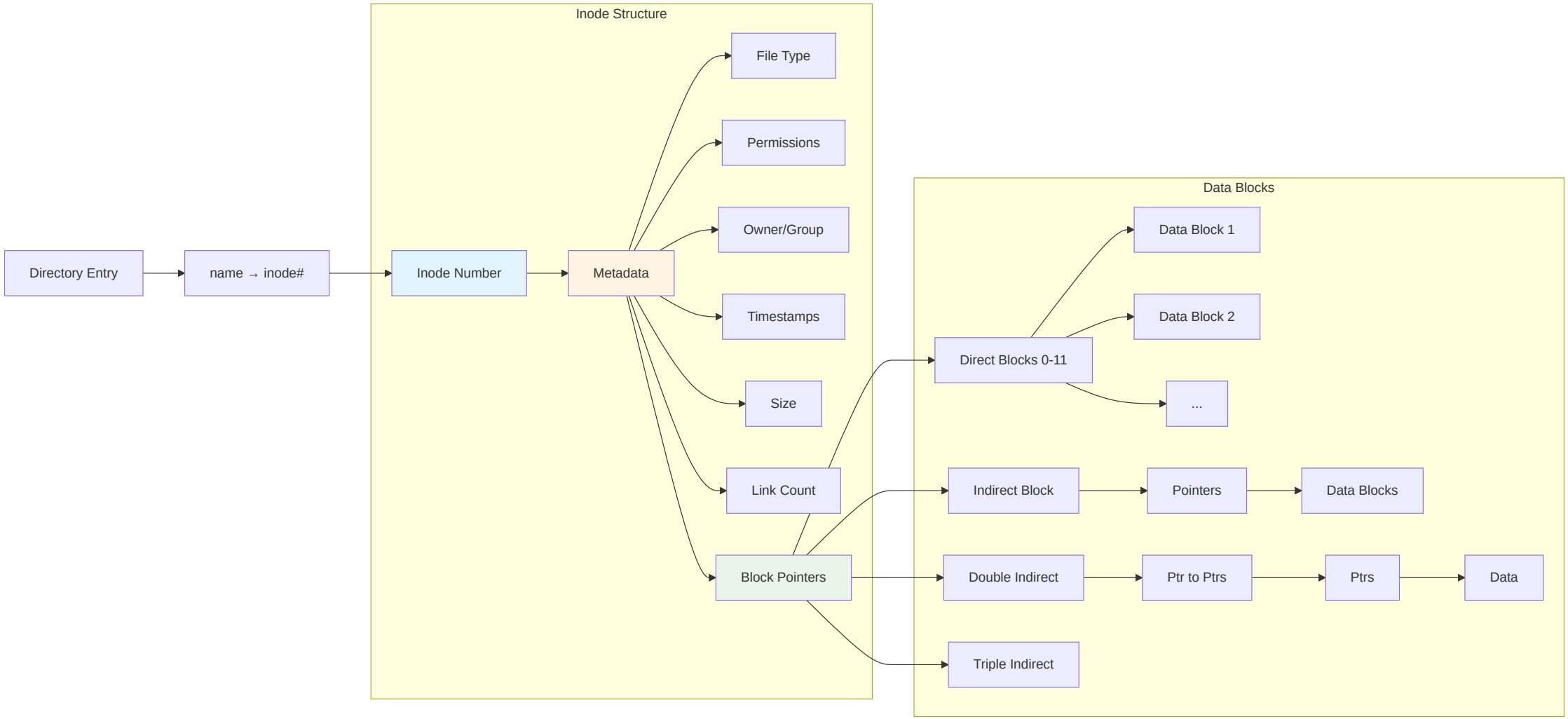


VFS provides uniform API across all file systems

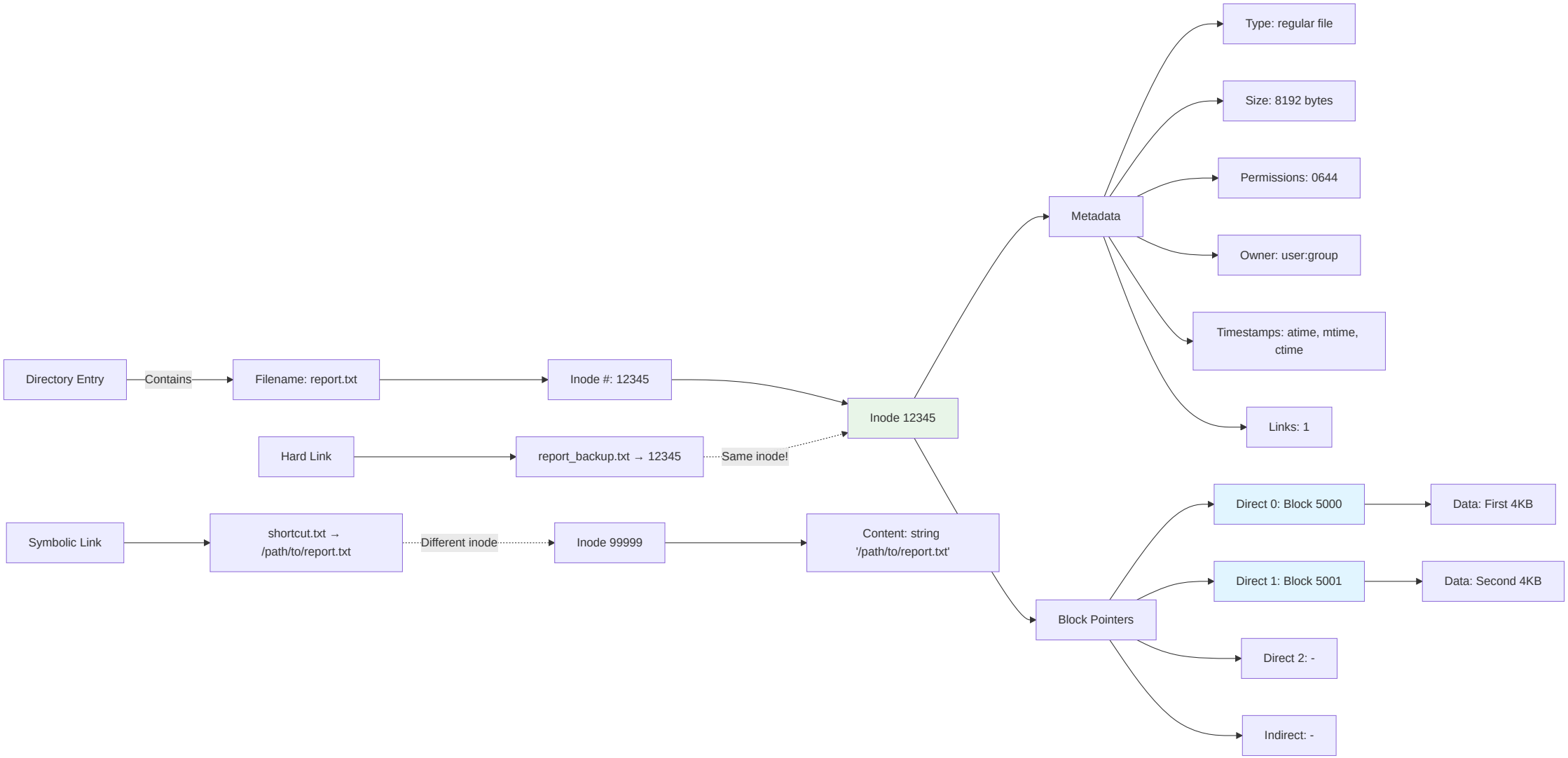
File I/O Layers



Inodes: The Heart of File Systems



Directory Structure

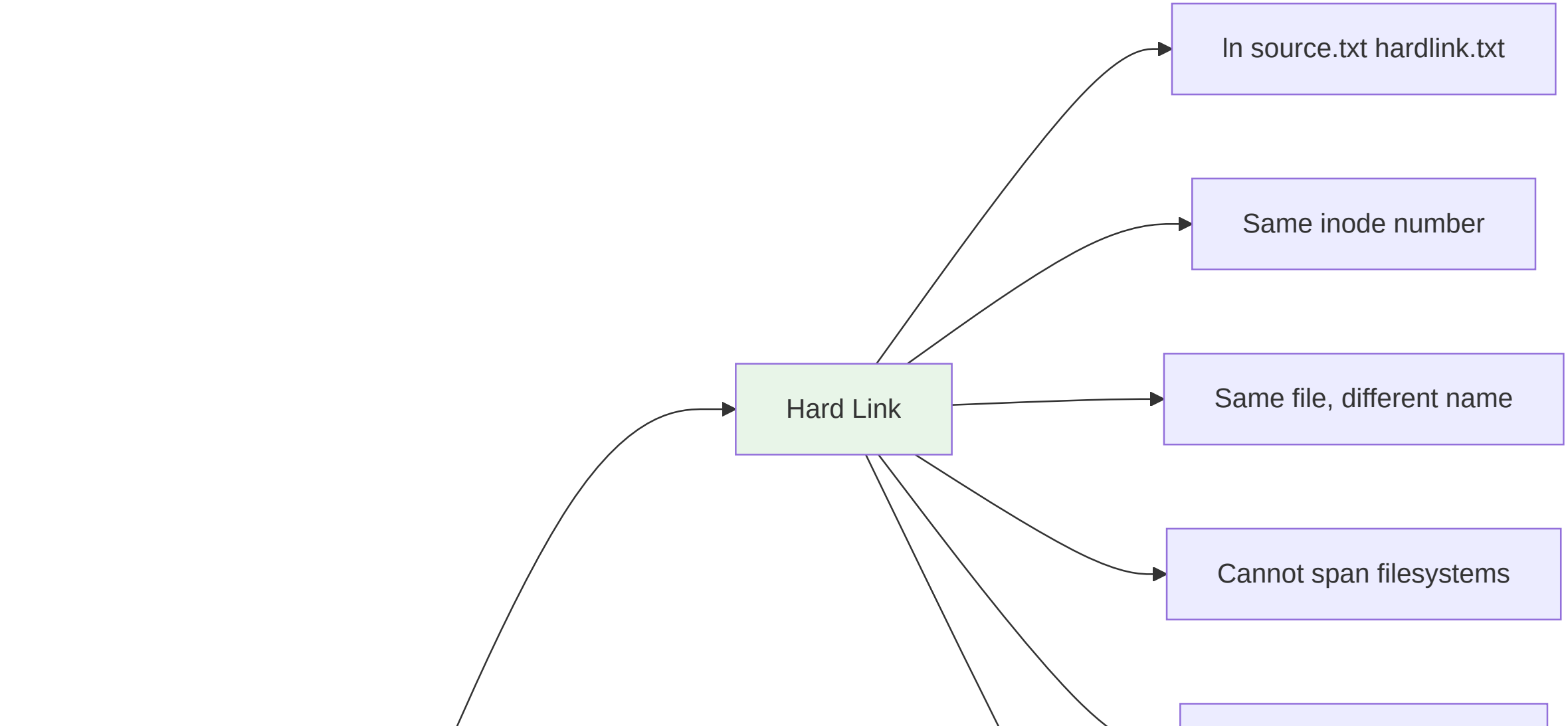


Inode vs Filename

```
# Filename is just a pointer to inode
$ ls -li file.txt
1234567 -rw-r--r-- 1 user group 8192 Mar 15 10:30 file.txt
#^^^^^^ inode number

# Create hard link (same inode)
$ ln file.txt file_link.txt
$ ls -li file*.txt
1234567 -rw-r--r-- 2 user group 8192 Mar 15 10:30 file.txt
1234567 -rw-r--r-- 2 user group 8192 Mar 15 10:30 file_link.txt
#                  ^ link count increased

# Check inode details
$ stat file.txt
```

Deleting a file removes directory entry; inode freed when link count = 0

Hard Links vs Symbolic Links



ext4 File System

Modern Linux Default

- **Extents** - contiguous block allocation (less fragmentation)
- **Delayed allocation** - batch writes for better placement
- **Journal checksumming** - detect corruption
- **Large file support** - up to 16 TB
- **Large volume support** - up to 1 EB
- **Online defragmentation** - `e4defrag`
- **Fast fsck** - uninit_bg feature

```
# View ext4 info
tune2fs -l /dev/sda1
```

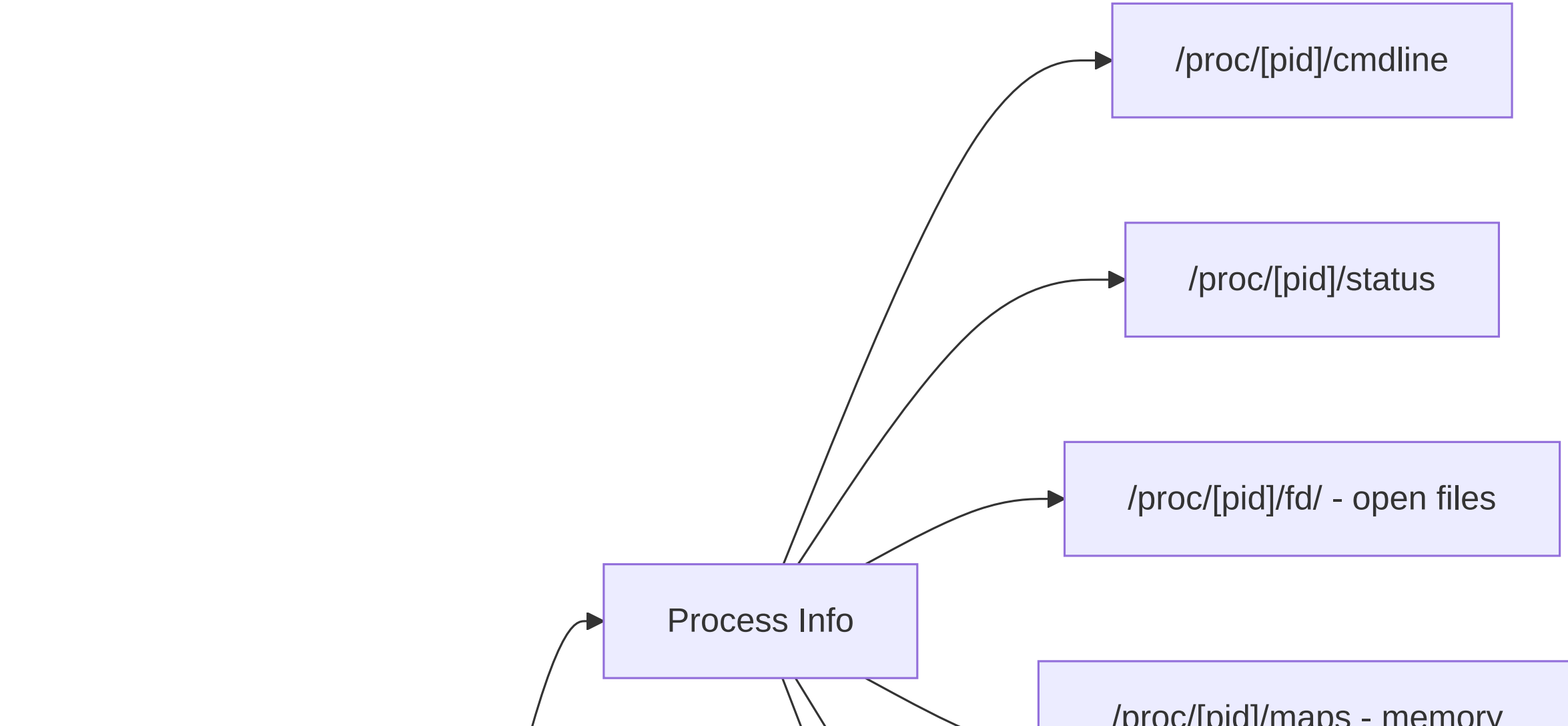
Journaling: Data Protection

```
Write operation flow:
1. Write to journal (commit record)
2. Write to actual location
3. Mark journal entry complete (checkpoint)

On crash:
- Replay journal entries
- File system consistent in seconds (not hours!)
```

Journal Modes

- **data=journal** - Log data + metadata (slowest, safest)
- **data=ordered** - Write data before metadata (default)
- **data=writeback** - No ordering (fastest, less safe)

/proc and /sys Filesystems

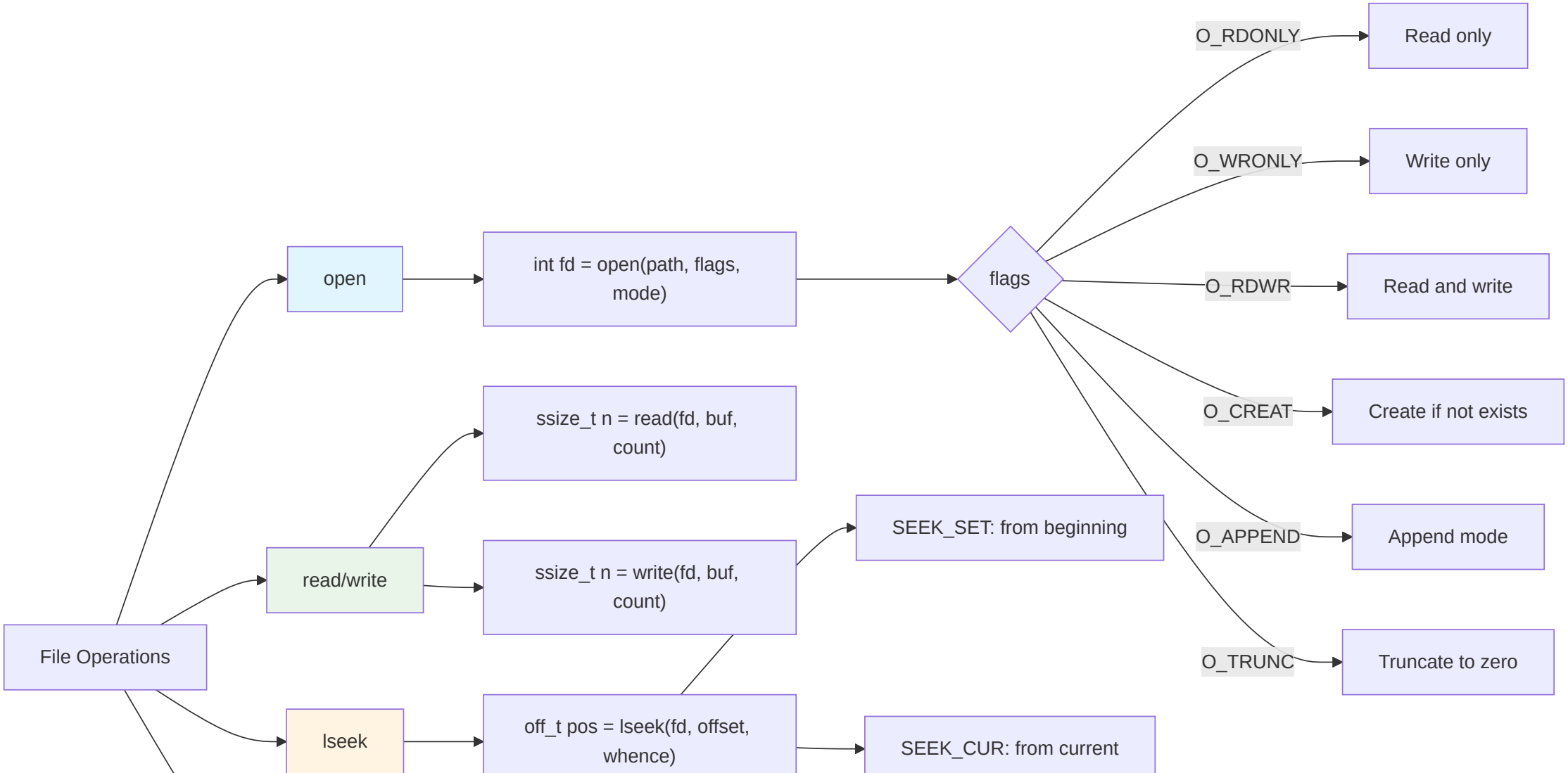


Exploring /proc

```
# Process info
cat /proc/$$/cmdline    # Current shell command
cat /proc/$$/status     # Process status
ls -l /proc/$$/fd/      # Open file descriptors
cat /proc/$$/maps       # Memory mappings

# System info
cat /proc/cpuinfo       # CPU details
cat /proc/meminfo       # Memory usage
cat /proc/loadavg       # System load
cat /proc/version       # Kernel version

# Kernel parameters (read/write)
cat /proc/sys/kernel/pid_max
echo 4194304 > /proc/sys/kernel/pid_max  # (as root)
```

Section 2: File I/O APIs

File I/O API Overview



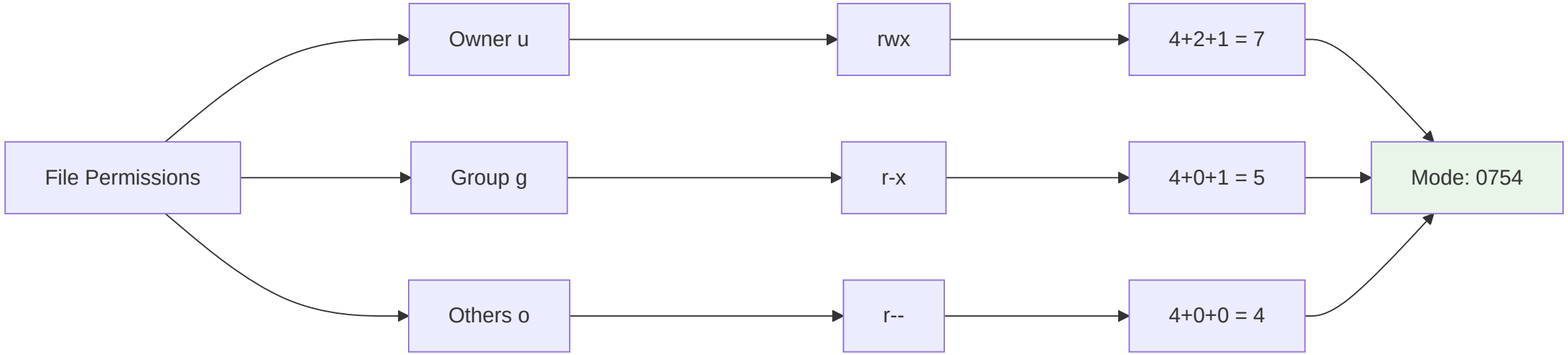
Opening Files: open()

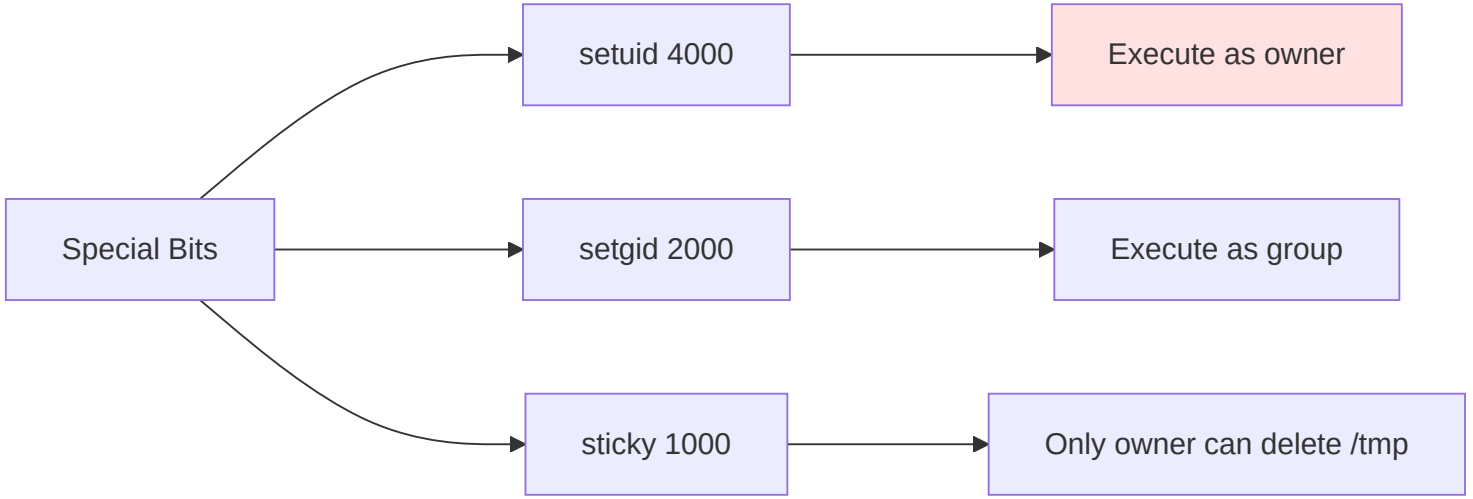
```c
#include <fcntl.h>
#include <sys/stat.h>

int open(const char *pathname, int flags, mode_t mode);

// Read only
int fd = open("/etc/passwd", O_RDONLY);

// Write only, create if not exists, truncate if exists
int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

// Read/write, create if not exists, append mode
int fd = open("log.txt", O_RDWR | O_CREAT | O_APPEND, 0644);

// Returns -1 on error
if (fd == -1) {
    perror("open");
    return EXIT_FAILURE;
}
```

Open Flags Reference

```
// Access modes (mutually exclusive)
O_RDONLY    // Read only
O_WRONLY    // Write only
O_RDWR      // Read and write

// Creation flags
O_CREAT     // Create if not exists (requires mode)
O_EXCL      // Fail if exists (with O_CREAT) - atomic test-and-create
O_TRUNC     // Truncate to zero length

// File status flags
O_APPEND    // Append mode (atomic seek to end + write)
O_NONBLOCK  // Non-blocking I/O
O_SYNC      // Synchronous writes (wait for physical I/O)
O_DIRECT    // Direct I/O (bypass page cache)
O_CLOEXEC   // Close on exec
```

File Permissions (mode)





Reading from Files: read()

```c
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);

// Example
char buffer[4096];
ssize_t bytes_read = read(fd, buffer, sizeof(buffer));

if (bytes_read == -1) {
    perror("read");
    return -1;
} else if (bytes_read == 0) {
    // End of file
    printf("EOF reached\n");
} else {
    // bytes_read may be less than count!
    printf("Read %zd bytes\n", bytes_read);
}
```

Writing to Files: write()

```c
#include <unistd.h>

ssize_t write(int fd, const void *buf, size_t count);

// Example
const char *message = "Hello, World!\n";
ssize_t bytes_written = write(fd, message, strlen(message));

if (bytes_written == -1) {
    perror("write");
    return -1;
}

if (bytes_written < strlen(message)) {
    // Partial write! Must retry
    printf("Partial write: %zd of %zu bytes\n",
           bytes_written, strlen(message));
}
```

Handling Partial I/O

```c
// Helper function: read exactly N bytes
ssize_t read_full(int fd, void *buf, size_t count) {
    size_t total = 0;
    ssize_t n;

    while (total < count) {
        n = read(fd, (char *)buf + total, count - total);
        if (n == -1) {
            if (errno == EINTR) continue;  // Interrupted, retry
            return -1;
        }
        if (n == 0) break;  // EOF
        total += n;
    }

    return total;
}

// Similarly for write_full()
ssize_t write_full(int fd, const void *buf, size_t count) {
    size_t total = 0;
    ssize_t n;

    while (total < count) {
        n = write(fd, (const char *)buf + total, count - total);
        if (n == -1) {
            if (errno == EINTR) continue;
            return -1;
        }
        total += n;
    }

    return total;
}
```

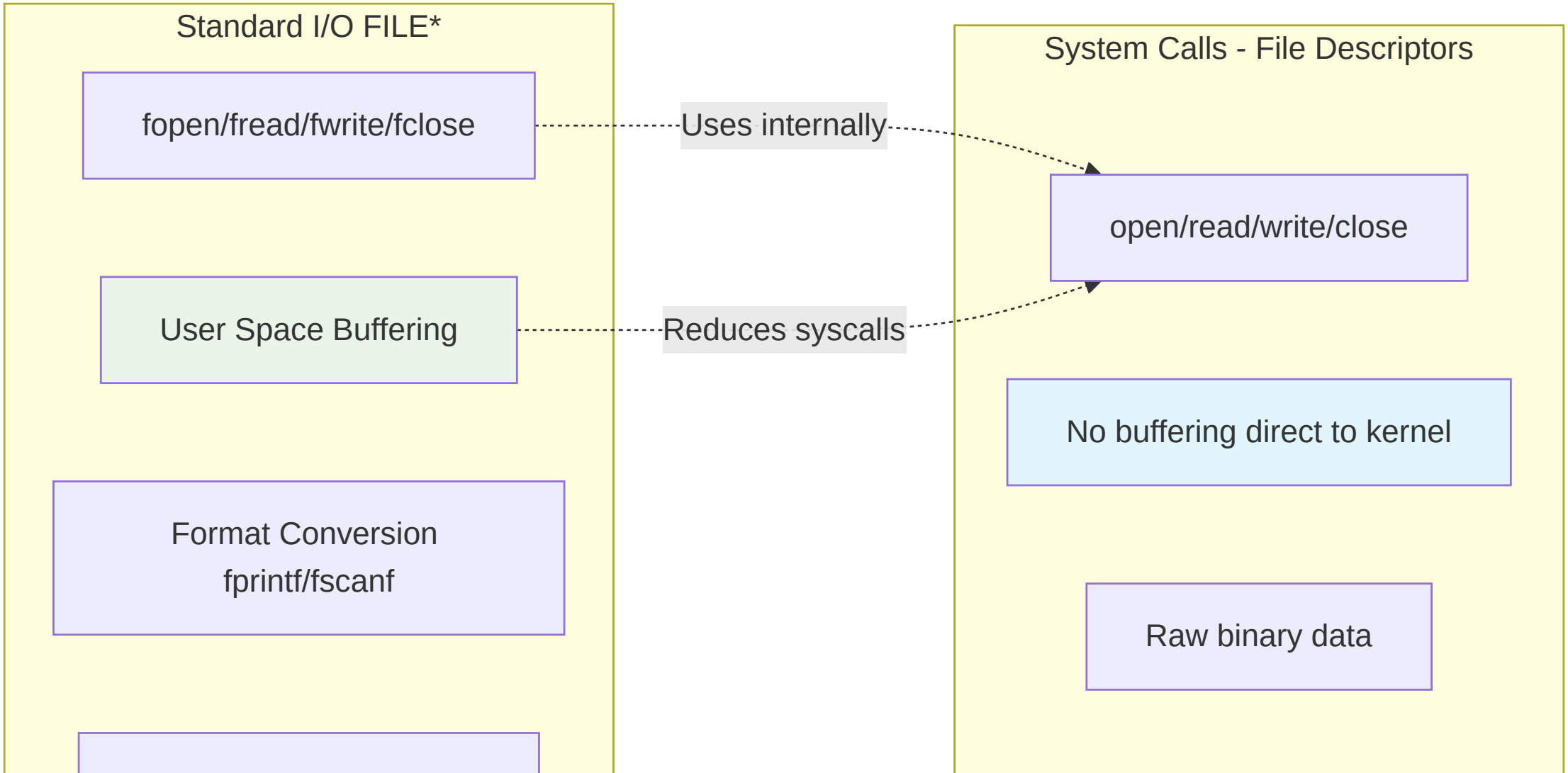
Seeking: lseek()

```c
#include <unistd.h>

off_t lseek(int fd, off_t offset, int whence);

// whence options:
SEEK_SET  // From beginning of file
SEEK_CUR  // From current position
SEEK_END  // From end of file

// Examples
lseek(fd, 0, SEEK_SET);        // Go to beginning
lseek(fd, 0, SEEK_END);        // Go to end (get file size)
lseek(fd, 100, SEEK_SET);      // Go to byte 100
lseek(fd, -10, SEEK_CUR);      // Back 10 bytes
lseek(fd, -20, SEEK_END);      // 20 bytes before EOF

// Get current position
off_t pos = lseek(fd, 0, SEEK_CUR);

// Get file size
off_t size = lseek(fd, 0, SEEK_END);
```

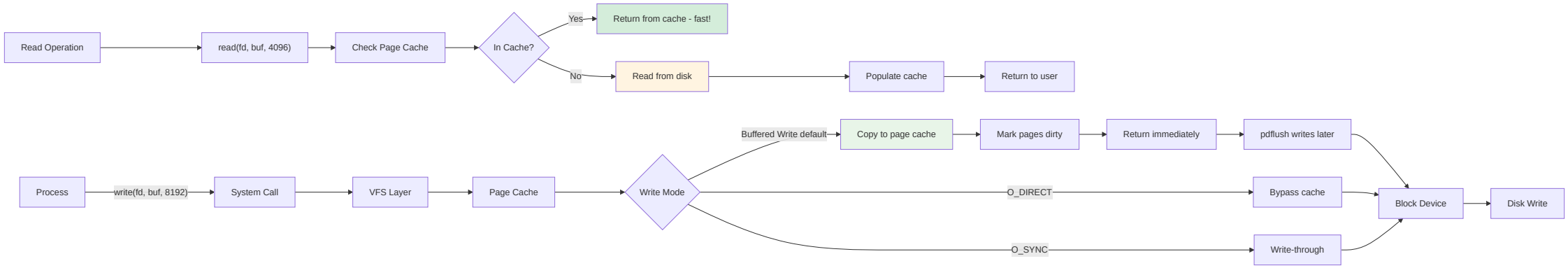
Complete File Copy Example

```c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int copy_file(const char *src, const char *dst) {
    int src_fd = open(src, O_RDONLY);
    if (src_fd == -1) {
        perror("open source");
        return -1;
    }

    int dst_fd = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dst_fd == -1) {
        perror("open destination");
        close(src_fd);
        return -1;
    }

    char buffer[8192];
    ssize_t n;

    while ((n = read(src_fd, buffer, sizeof(buffer))) > 0) {
        if (write_full(dst_fd, buffer, n) != n) {
            perror("write");
            goto cleanup;
        }
    }

    if (n == -1) {
        perror("read");
    }

cleanup:
    close(src_fd);
    close(dst_fd);
    return (n == -1) ? -1 : 0;
}
```

Standard I/O vs System Calls



Buffered I/O Performance



Standard I/O (FILE *)

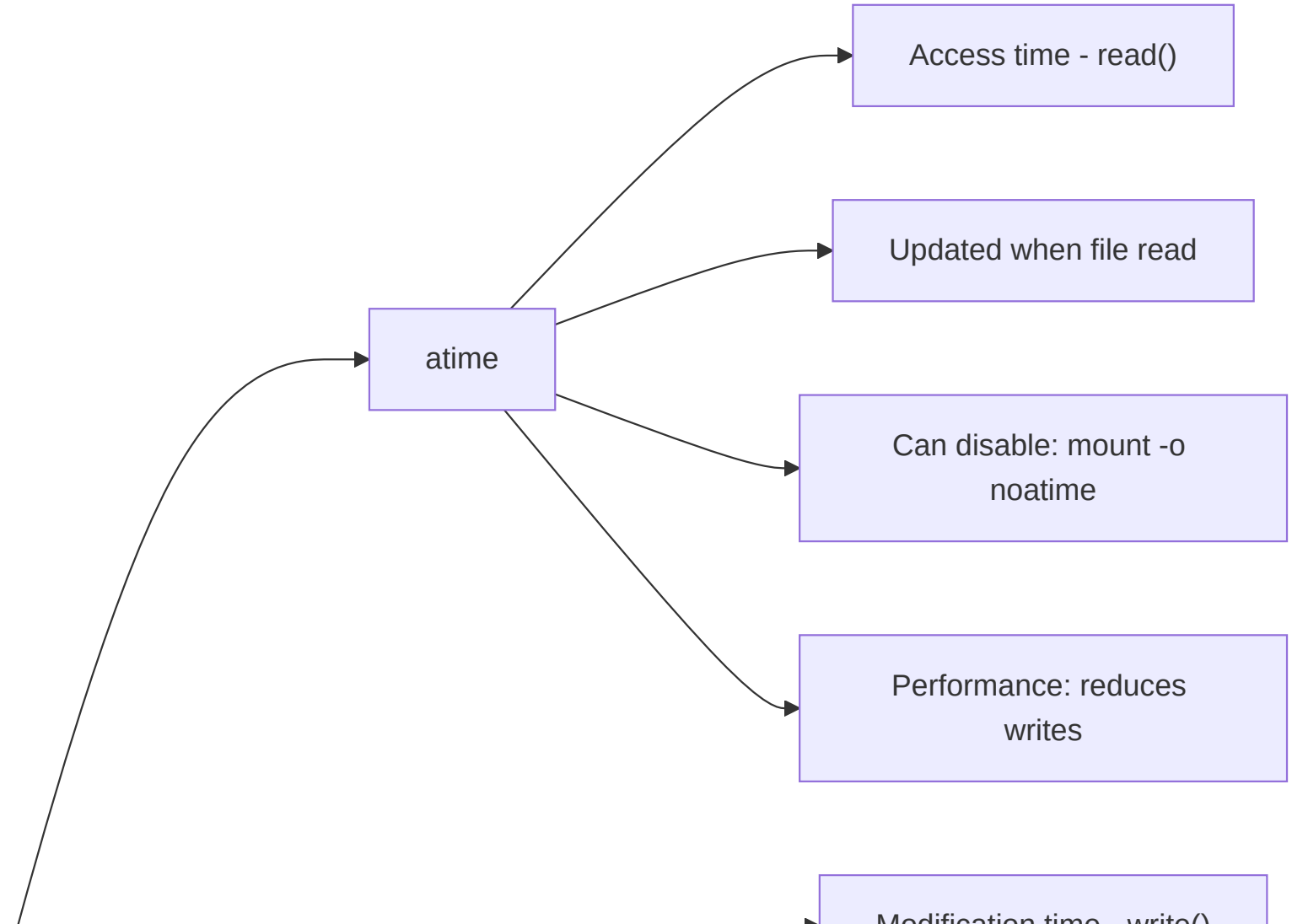
```c
#include <stdio.h>

// Open
FILE *fp = fopen("data.txt", "r");  // "r", "w", "a", "r+", "w+", "a+"
if (!fp) {
    perror("fopen");
    return -1;
}

// Read
char buffer[256];
if (fgets(buffer, sizeof(buffer), fp) != NULL) {
    printf("Line: %s", buffer);
}

// Formatted read
int num;
char name[50];
fscanf(fp, "%d %s", &num, name);

// Write
fprintf(fp, "Number: %d\n", 42);
fputs("Hello\n", fp);

// Binary I/O
struct record data;
fread(&data, sizeof(data), 1, fp);
fwrite(&data, sizeof(data), 1, fp);

// Close
fclose(fp);
```

Buffering Modes

```c
#include <stdio.h>

// Fully buffered (default for files)
setvbuf(fp, NULL, _IOFBF, 8192);

// Line buffered (default for terminals)
setvbuf(fp, NULL, _IOLBF, 0);

// Unbuffered (stderr default)
setvbuf(fp, NULL, _IONBF, 0);

// Custom buffer
char buffer[16384];
setvbuf(fp, buffer, _IOFBF, sizeof(buffer));

// Flush buffer manually
fflush(fp);
```


Use FILE for portability and convenience; use fd for performance

File Timestamps



Section 3: Advanced File Operations

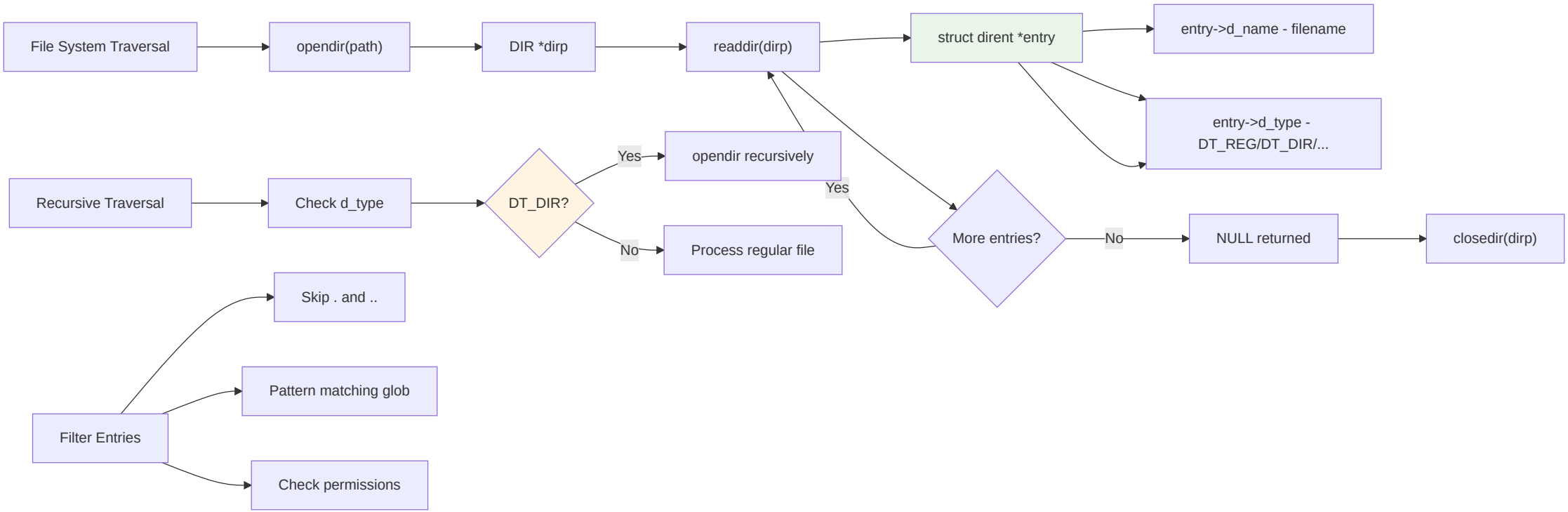
File Metadata: stat(), fstat(), lstat()

```c
#include <sys/stat.h>

struct stat sb;

// stat: follow symlinks
if (stat("/path/to/file", &sb) == -1) {
    perror("stat");
    return -1;
}

// lstat: don't follow symlinks
lstat("/path/to/symlink", &sb);

// fstat: from file descriptor
int fd = open("file.txt", O_RDONLY);
fstat(fd, &sb);

// struct stat members
sb.st_mode     // File type and permissions
sb.st_size     // File size in bytes
sb.st_blocks   // Number of 512B blocks
sb.st_uid      // Owner user ID
sb.st_gid      // Owner group ID
sb.st_atime    // Access time
sb.st_mtime    // Modification time
sb.st_ctime    // Change time (metadata)
sb.st_ino      // Inode number
sb.st_nlink    // Number of hard links
```

Testing File Types

```c
#include <sys/stat.h>

struct stat sb;
stat(path, &sb);

// Test file type macros
if (S_ISREG(sb.st_mode))  printf("Regular file\n");
if (S_ISDIR(sb.st_mode))  printf("Directory\n");
if (S_ISLNK(sb.st_mode))  printf("Symbolic link\n");
if (S_ISCHR(sb.st_mode))  printf("Character device\n");
if (S_ISBLK(sb.st_mode))  printf("Block device\n");
if (S_ISFIFO(sb.st_mode)) printf("FIFO/pipe\n");
if (S_ISSOCK(sb.st_mode)) printf("Socket\n");

// Check permissions
if (sb.st_mode & S_IRUSR) printf("User can read\n");
if (sb.st_mode & S_IWUSR) printf("User can write\n");
if (sb.st_mode & S_IXUSR) printf("User can execute\n");

// Check access using access()
if (access(path, R_OK | W_OK) == 0) {
    printf("Readable and writable\n");
}
```

Directory Iteration



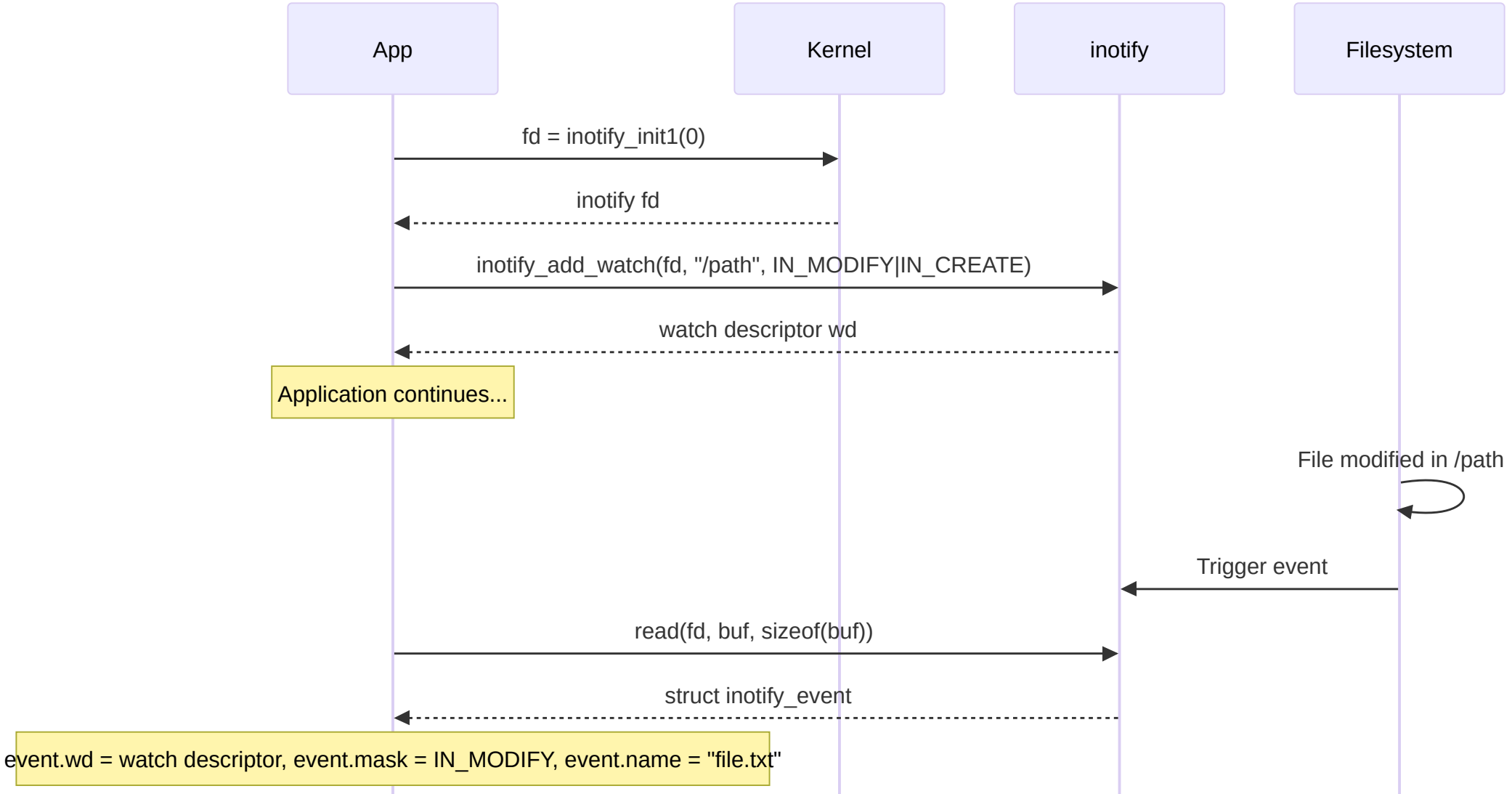
Directory Operations

```c
#include <dirent.h>
#include <sys/stat.h>

void list_directory(const char *path) {
    DIR *dirp = opendir(path);
    if (!dirp) {
        perror("opendir");
        return;
    }

    struct dirent *entry;
    while ((entry = readdir(dirp)) != NULL) {
        // Skip . and ..
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0) {
            continue;
        }

        printf("%s ", entry->d_name);

        // Check type
        if (entry->d_type == DT_DIR) {
            printf("(directory)\n");
        } else if (entry->d_type == DT_REG) {
            printf("(file)\n");
        } else if (entry->d_type == DT_LNK) {
            printf("(symlink)\n");
        } else {
            printf("(other)\n");
        }
    }

    closedir(dirp);
}
```

Recursive Directory Traversal

```c
#include <sys/stat.h>
#include <dirent.h>
#include <string.h>

void traverse_recursive(const char *path, int depth) {
    DIR *dirp = opendir(path);
    if (!dirp) return;

    struct dirent *entry;
    while ((entry = readdir(dirp)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0) {
            continue;
        }

        // Print with indentation
        for (int i = 0; i < depth; i++) printf("  ");
        printf("%s\n", entry->d_name);

        // Recurse into subdirectories
        if (entry->d_type == DT_DIR) {
            char subpath[PATH_MAX];
            snprintf(subpath, sizeof(subpath), "%s/%s", path, entry->d_name);
            traverse_recursive(subpath, depth + 1);
        }
    }

    closedir(dirp);
}
```

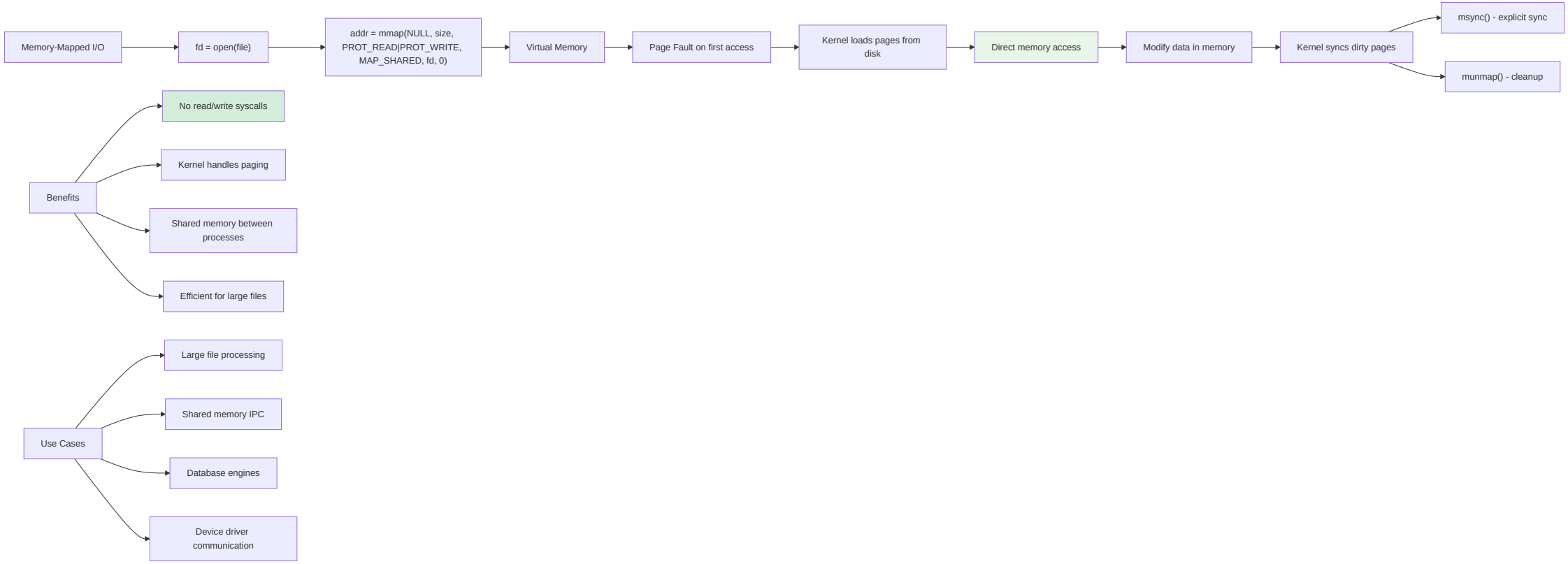
File System Monitoring: inotify



Using inotify

```c
#include <sys/inotify.h>

int fd = inotify_init1(IN_NONBLOCK);
if (fd == -1) {
    perror("inotify_init1");
    return -1;
}

// Add watch
int wd = inotify_add_watch(fd, "/path/to/watch",
                           IN_MODIFY | IN_CREATE | IN_DELETE);
if (wd == -1) {
    perror("inotify_add_watch");
    return -1;
}

// Read events
char buffer[4096] __attribute__((aligned(__alignof__(struct inotify_event))));

while (1) {
    ssize_t len = read(fd, buffer, sizeof(buffer));
    if (len == -1 && errno != EAGAIN) {
        perror("read");
        break;
    }

    for (char *ptr = buffer; ptr < buffer + len;) {
        struct inotify_event *event = (struct inotify_event *)ptr;

        if (event->mask & IN_CREATE)
            printf("Created: %s\n", event->name);
        if (event->mask & IN_DELETE)
            printf("Deleted: %s\n", event->name);
        if (event->mask & IN_MODIFY)
            printf("Modified: %s\n", event->name);

        ptr += sizeof(struct inotify_event) + event->len;
    }
}

inotify_rm_watch(fd, wd);
close(fd);
```

Memory-Mapped I/O



Using mmap()

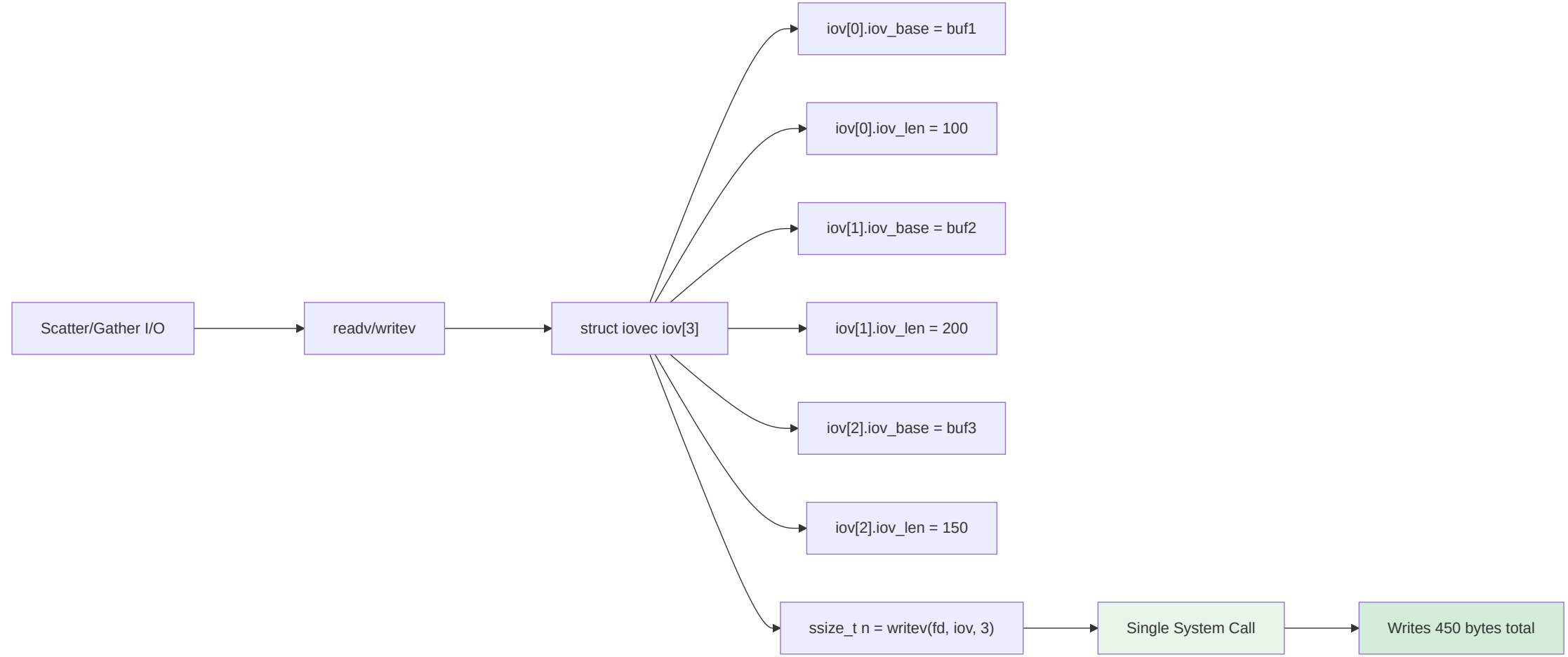
```c
#include <sys/mman.h>
#include <fcntl.h>

int fd = open("data.bin", O_RDWR);
if (fd == -1) {
    perror("open");
    return -1;
}

// Get file size
struct stat sb;
fstat(fd, &sb);

// Map file into memory
void *addr = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
if (addr == MAP_FAILED) {
    perror("mmap");
    close(fd);
    return -1;
}

// Access as memory
unsigned char *data = (unsigned char *)addr;
data[0] = 0xFF;  // Modifies file!

// Sync to disk (optional, kernel does this automatically)
msync(addr, sb.st_size, MS_SYNC);

// Unmap
munmap(addr, sb.st_size);
close(fd);
```

mmap() Flags

```
// Protection flags (prot)
PROT_READ    // Pages may be read
PROT_WRITE   // Pages may be written
PROT_EXEC    // Pages may be executed
PROT_NONE    // Pages may not be accessed

// Mapping flags (flags)
MAP_SHARED   // Share mapping (changes visible to other processes)
MAP_PRIVATE  // Copy-on-write private mapping
MAP_ANONYMOUS // Not backed by file (for malloc-like allocation)
MAP_FIXED    // Use exact address (dangerous!)
MAP_POPULATE // Pre-populate page tables (avoid page faults)
MAP_LOCKED   // Lock pages in memory (like mlock())

// Example: shared memory between processes
void *shm = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                 MAP_SHARED | MAP_ANONYMOUS, -1, 0);
```

Section 4: Advanced I/O Techniques

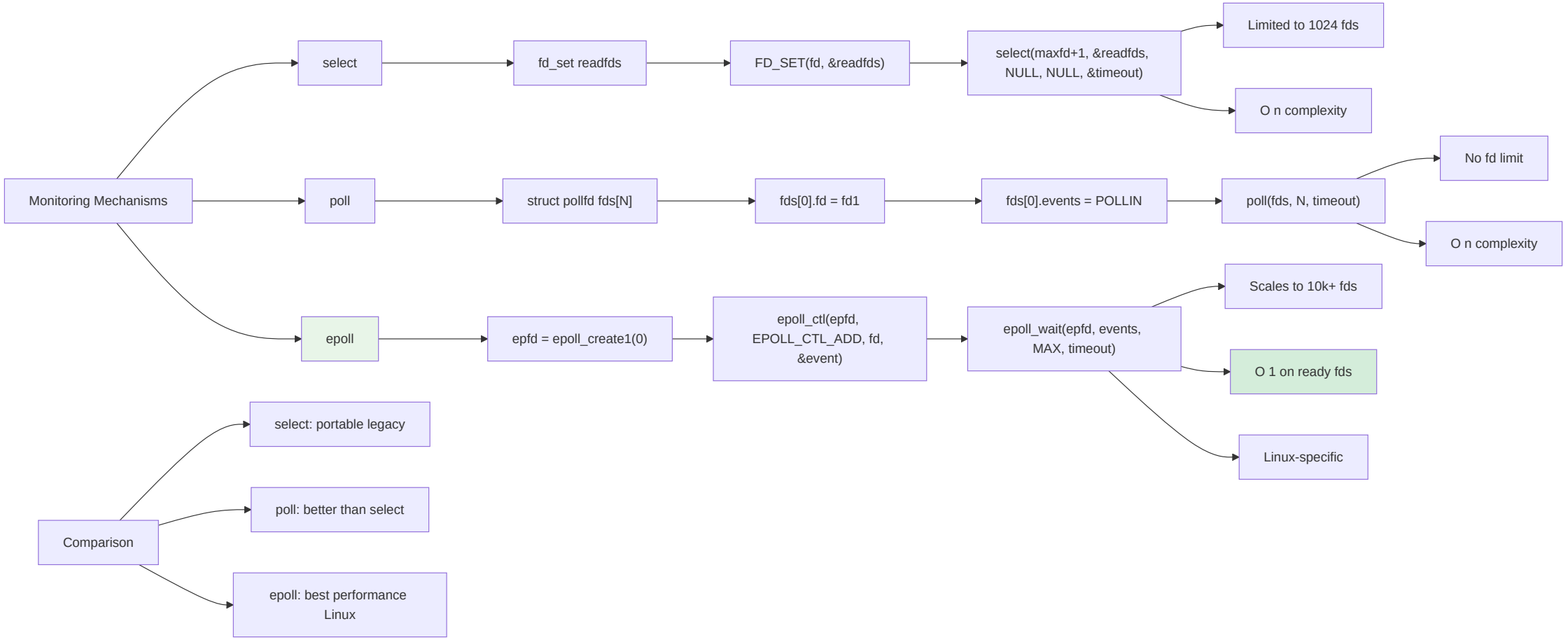
Scatter/Gather I/O



Using readv() and writev()

```c
#include <sys/uio.h>

// Scatter read: read into multiple buffers
struct iovec iov[3];
char buf1[100], buf2[200], buf3[150];

iov[0].iov_base = buf1;
iov[0].iov_len = sizeof(buf1);
iov[1].iov_base = buf2;
iov[1].iov_len = sizeof(buf2);
iov[2].iov_base = buf3;
iov[2].iov_len = sizeof(buf3);

ssize_t n = readv(fd, iov, 3);  // Single syscall!

// Gather write: write from multiple buffers
const char *header = "HTTP/1.1 200 OK\r\n";
const char *body = "<html>...</html>";

iov[0].iov_base = (void *)header;
iov[0].iov_len = strlen(header);
iov[1].iov_base = (void *)body;
iov[1].iov_len = strlen(body);

n = writev(socket_fd, iov, 2);  // Atomic write
```

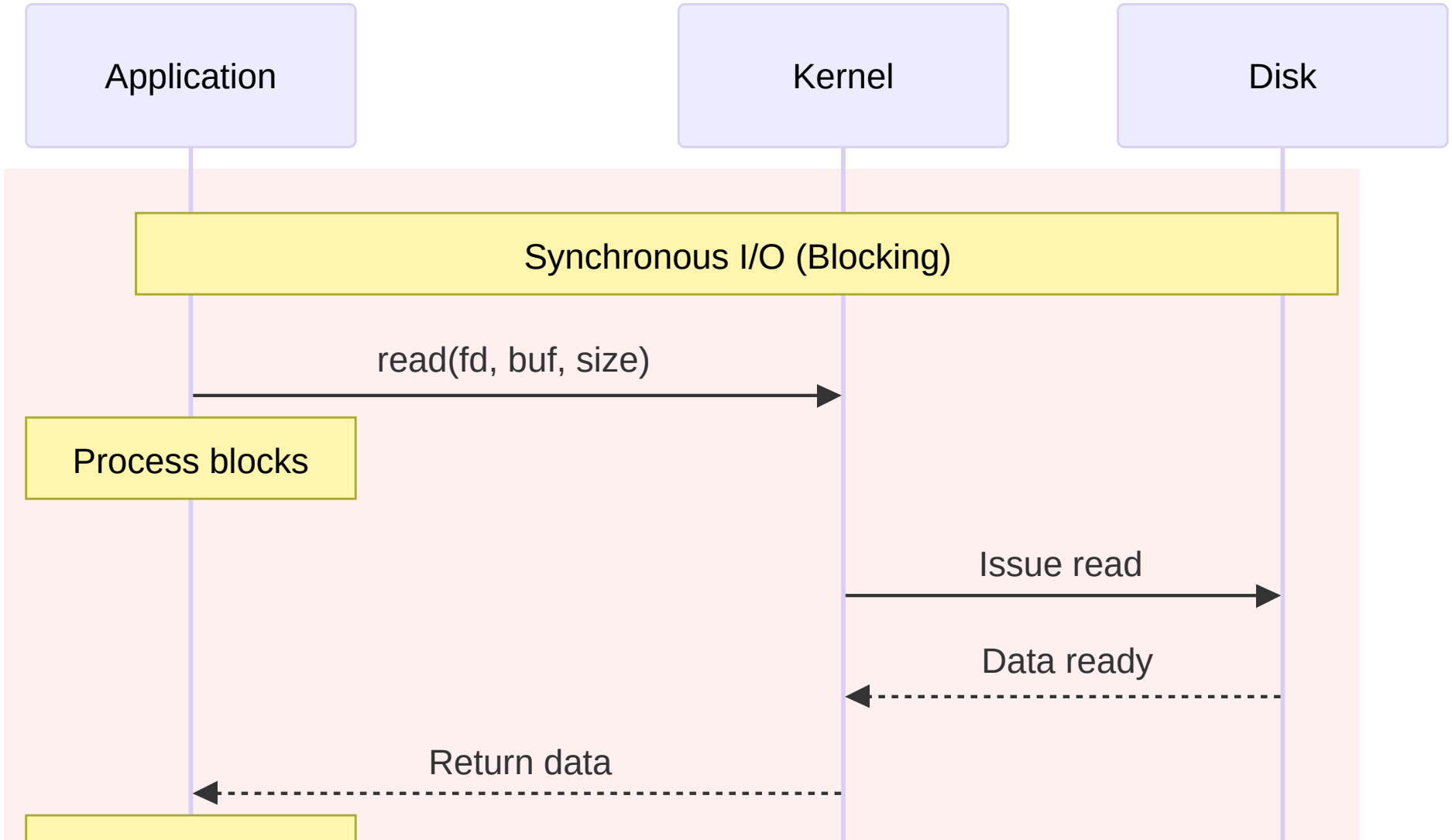
I/O Multiplexing



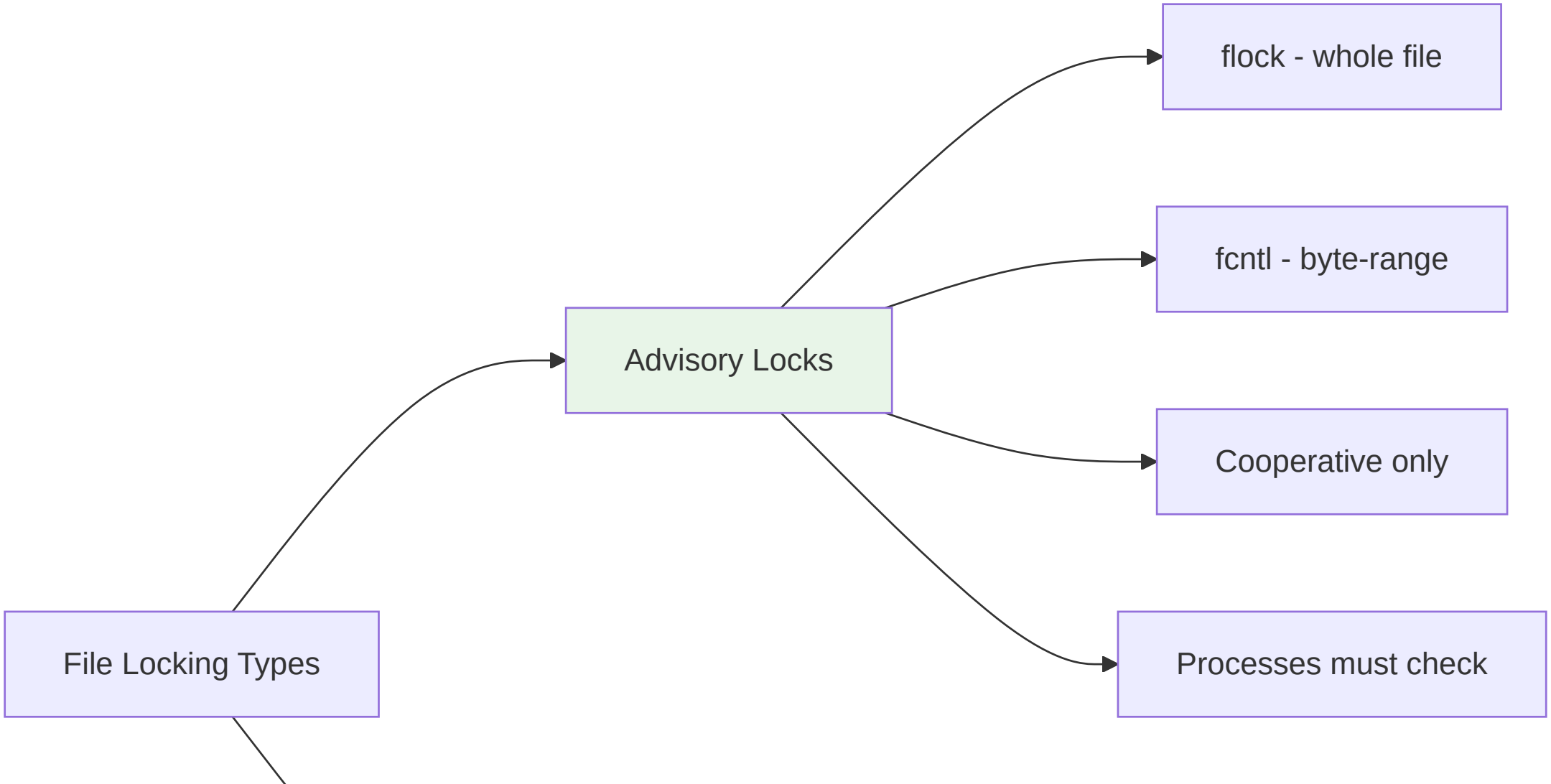
epoll Example

```c
#include <sys/epoll.h>

int epfd = epoll_create1(0);
if (epfd == -1) {
    perror("epoll_create1");
    return -1;
}

// Add file descriptor to epoll
struct epoll_event event;
event.events = EPOLLIN;  // Monitor for read
event.data.fd = fd;

if (epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &event) == -1) {
    perror("epoll_ctl");
    return -1;
}

// Wait for events
struct epoll_event events[MAX_EVENTS];
int nfds = epoll_wait(epfd, events, MAX_EVENTS, -1);

for (int i = 0; i < nfds; i++) {
    if (events[i].events & EPOLLIN) {
        int ready_fd = events[i].data.fd;
        // Read from ready_fd
    }
}
```

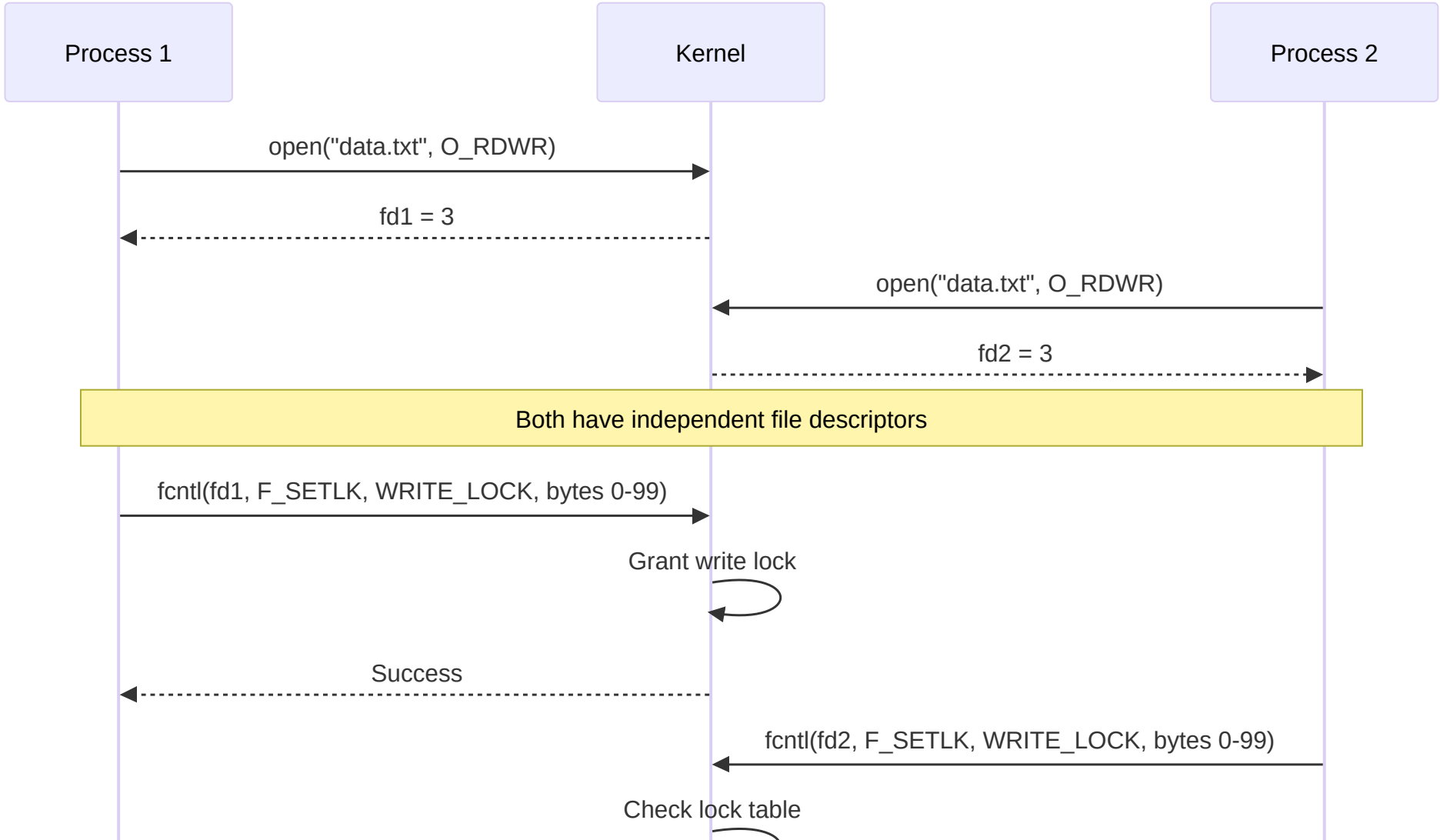
Asynchronous I/O



File Locking



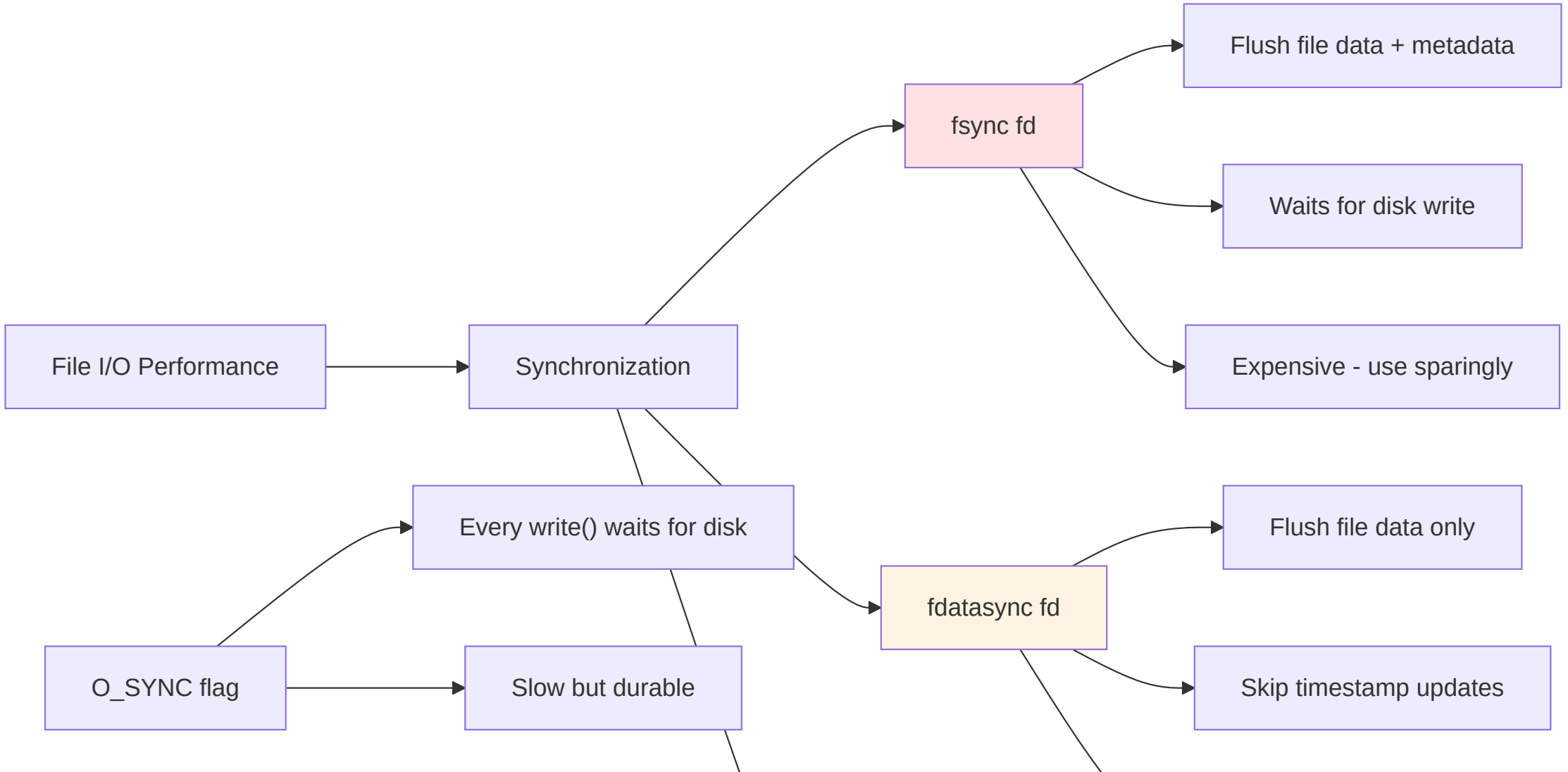
File Locking Sequence



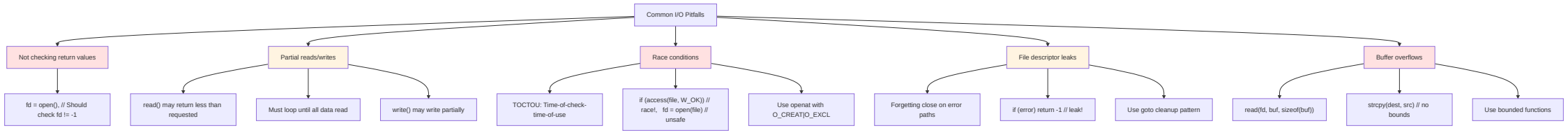
fcntl() Locking Example

```c
#include <fcntl.h>

int fd = open("data.txt", O_RDWR);

// Setup lock structure
struct flock fl;
fl.l_type = F_WRLCK;    // Write (exclusive) lock
fl.l_whence = SEEK_SET;
fl.l_start = 0;         // Lock from beginning
fl.l_len = 0;           // 0 means until EOF

// Try to acquire lock (non-blocking)
if (fcntl(fd, F_SETLK, &fl) == -1) {
    if (errno == EACCES || errno == EAGAIN) {
        printf("File is locked by another process\n");
    } else {
        perror("fcntl");
    }
    return -1;
}

// Critical section: modify file
// ...

// Release lock
fl.l_type = F_UNLCK;
fcntl(fd, F_SETLK, &fl);
```

Performance: Sync Operations



Common I/O Pitfalls



Day 3 Summary

What We Covered

✅ **File systems** - VFS, inodes, ext4, journaling
✅ **File I/O APIs** - open, read, write, lseek, close
✅ **Standard I/O** - FILE*, buffering, fopen/fread/fwrite
✅ **Advanced operations** - stat, directories, inotify
✅ **mmap** - Memory-mapped files for efficient I/O
✅ **File locking** - Coordination with flock/fcntl
✅ **Advanced I/O** - scatter/gather, epoll, async

Key Takeaways

Critical Concepts

1. **Check all return values** - especially open() and write()
2. **Handle partial reads/writes** - loop until complete
3. **Use mmap() for large files** - avoid excessive syscalls
4. **File locking is advisory** - processes must cooperate
5. **Buffer I/O when possible** - reduces system call overhead
6. **Always close file descriptors** - avoid resource leaks

File I/O errors are often ignored - don't be that developer!

Common Mistakes

❌ Not checking open() return value
❌ Assuming read() returns exactly what you asked for
❌ Ignoring errno after errors
❌ Using strcpy() instead of strncpy()
❌ TOCTOU bugs (check-then-use race conditions)
❌ Forgetting to close() on error paths
❌ Not using O_CLOEXEC (file descriptors leak to child processes)

Write defensive code: check, validate, handle errors

Hands-On Exercises

Lab Time: ~4 Hours

Navigate to: `exercises/03-file-operations-mmap.md`

You will:

1. Implement file copy with proper error handling
2. Build directory tree traversal utility
3. Create file system monitor with inotify
4. Use mmap() to process large files efficiently
5. Implement file locking for multi-process coordination
6. Compare performance: read/write vs mmap

Tomorrow: Day 4

Processes, IPCand Signals

- Process lifecycle (fork, exec, wait)
- Process creation and termination
- Inter-process communication (pipes, FIFOs, shared memory)
- Signals and signal handling
- Process groups and sessions

Exercises connect: Use files for IPC and data exchange

Questions?

Resources

- File I/O: `man 2 open`, `man 2 read`, `man 2 write`
- Standard I/O: `man 3 fopen`, `man 3 fread`
- mmap: `man 2 mmap`
- inotify: `man 7 inotify`
- File locking: `man 2 fcntl`, `man 2 flock`

Office hours: After lab session

Additional Resources

Documentation

- Linux File System Hierarchy
- ext4 Documentation
- inotify(7) man page
- POSIX I/O Reference

Books

- "The Linux Programming Interface" by Kerrisk (Chapters 4-5, 13-18)
- "Advanced Programming in the UNIX Environment" by Stevens

Break!

Next Up: Hands-On Lab

☕ Take 10 minutes, then start implementing real file I/O code

Remember: The filesystem is where data lives - master it!